Bài thực hành 4

Hàm, tham số và mô-đun

Hà Minh Tuấn

Khoa Toán — Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Phần A: 2,5 tiết tại lớp · Phần B: làm ở nhà

Mục tiêu

- Viết hàm có `return`, trả về nhiều giá trị, có docstring và chú thích kiểu.
- Dùng đúng tham số mặc định, tham số chỉ nhận theo tên (`*`), `*args`, `**kwargs`; tránh giá trị mặc định khả biến.
- Hiểu phạm vi biến: hằng số cấu hình đọc từ bên trong hàm; hàm nhận qua tham số và trả kết quả ra, không ghi vào biến bên ngoài.
- Dùng các hàm dựng sẵn `sorted`, `any`, `all` thay cho vòng lặp tự viết.
- Chia một chương trình thành mô-đun chứa các hàm và chương trình chính gọi lại mô-đun đó.

Bài này gồm hai phần

Phần	Làm ở đâu	Cách viết
A. Mô-đun thống kê điểm thi	Tại lớp, trên máy của Trường. 2,5 tiết.	Gõ mã mẫu, rồi viết các hàm tương tự trong `xu_ly_hs.py`; mỗi mục có kết quả để đối chiếu.
B. Bảng so sánh nhóm học sinh	Ở nhà, tự làm. **Có tính điểm.**	Chỉ nêu *yêu cầu* và vài gợi ý, không hướng dẫn từng bước.

Chuẩn bị

Mở sẵn slide **Bài 5 — Hàm** để tra cứu.

Tải từ Classroom về hai tệp: `xu_ly_hs.py` và `StudentsPerformance.csv` (cùng bộ dữ liệu của Bài thực hành 3).

Mục lục

Mục tiêu **1**

PHẦN A — LÀM TẠI LỚP **3**

1 Mở dự án (10 phút) **3**

2 Định nghĩa hàm và giá trị trả về (25 phút) **4**
2.1 Mã mẫu: `lay_cot` và `thong_ke` 4
2.2 Sinh viên tự thực hiện 4

3 Tham số (30 phút) **5**
3.1 Mã mẫu: `loc_hoc_sinh` 5
3.2 Sinh viên tự thực hiện 5

4 Phạm vi biến và hàm dựng sẵn (20 phút) **6**
4.1 Mã mẫu: `dat_ca_ba` 6
4.2 Sinh viên tự thực hiện 6

5 Tách chương trình thành mô-đun (35 phút) **6**
5.1 Mã mẫu: tệp `bao_cao.py` 7
5.2 Sinh viên tự thực hiện 7

Hết phần A — kiểm lại trước khi rời phòng máy **8**

PHẦN B — LÀM Ở NHÀ **9**

6 Yêu cầu **9**
6.1 Năm hàm dùng lại phần A 9
6.2 Hai hàm mở rộng 10

7 Gợi ý **11**

8 Nộp bài **11**

PHẦN A — LÀM TẠI LỚP

Mục	Nội dung	Thời lượng
A1	Mở dự án	10 phút
A2	Định nghĩa hàm và giá trị trả về	25 phút
A3	Tham số	30 phút
A4	Phạm vi biến và hàm dựng sẵn	20 phút
A5	Tách chương trình thành mô-đun	35 phút
	Tổng	**120 phút**

Phần A làm trong tệp `xu_ly_hs.py`; mục A5 tạo thêm tệp `bao_cao.py`. Từ A2 đến A5, mỗi mục gồm hai bước:

1. **Mã mẫu**: gõ lại thân hàm in trong đề vào đúng hàm có chú thích `# <-- MA MAU` trong `xu_ly_hs.py`, chạy, đối chiếu kết quả.
2. **Sinh viên tự thực hiện**: viết các hàm có chú thích `# <-- CAN VIET LAI THAN HAM`, cùng kiểu với mã mẫu nhưng làm một việc khác.

Mỗi lần sửa xong một hàm thì chạy `python xu_ly_hs.py` và đối chiếu với hộp **Mốc kiểm** của mục đó. Chưa đạt thì sửa lại hoặc hỏi giảng viên hướng dẫn trước khi làm mục sau.

1. Mở dự án (10 phút)

Mục đích

Tạo dự án tuần 4 và chạy thử mô-đun `xu_ly_hs.py`.

1. Trong VS Code: **File** → **Open Folder...**, tạo thư mục mới `python-tuan04` trên ổ `D:\`, rồi **Select Folder**.
2. Kéo hai tệp đã tải vào khung *Explorer*.
3. **Terminal** → **New Terminal**, tạo và kích hoạt môi trường ảo, rồi chạy thử:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python xu_ly_hs.py
```

Hàm `doc_du_lieu` đã viết sẵn, trả về danh sách 1000 `dict` với tám khoá như Bài thực hành 3: `gioi_tinh`, `nhom`, `hoc_van`, `an_trua`, `on_thi`, `toan`, `doc`, `viet`. Đầu tệp có ba hằng số:

```
TEP = "StudentsPerformance.csv"
MON = ("toan", "doc", "viet")
NGUONG_DAT = 50
```

Mốc kiểm

`python xu_ly_hs.py` chạy không báo lỗi và in ra các dòng `--- A2` đến `--- A5` với giá trị rỗng hoặc bằng 0, vì các hàm chưa được viết.

2. Định nghĩa hàm và giá trị trả về (25 phút)

Mục đích

Viết các hàm thống kê nhỏ, mỗi hàm một việc, có `return` và docstring; rồi ghép chúng thành một hàm trả về nhiều giá trị.

2.1 Mã mẫu: `lay_cot` và `thong_ke`

`lay_cot` lấy điểm một môn của mọi học sinh thành một danh sách. `thong_ke` trả về ba giá trị cùng lúc — thực chất là một `tuple`. Dòng đầu thân hàm là docstring; phần `: list` và `-> tuple` là chú thích kiểu.

xu_ly_hs.py — mã mẫu A2

```
def lay_cot(ds_hs, mon):
    """Lay diem mot mon cua moi hoc sinh thanh mot list."""
    cot = []
    for hs in ds_hs:
        cot.append(hs[mon])
    return cot


def thong_ke(ds: list) -> tuple:
    """Tra ve (nho nhat, lon nhat, trung binh lam tron 2) cua mot list so."""
    return min(ds), max(ds), round(sum(ds) / len(ds), 2)
```

Slide Bài 5: *Bốn thành phần của một hàm*; *Trả về nhiều giá trị*; *Viết cho người đọc sáu tháng sau*.

2.2 Sinh viên tự thực hiện

- `do_lech_chuan(ds)`: độ lệch chuẩn $\sqrt{\frac{1}{n}\sum(x-\bar{x})^2}$ của một danh sách số, làm tròn 2 chữ số. Căn bậc hai viết `** 0.5`.
- `ti_le_dat(ds, nguong=NGUONG_DAT)`: tỉ lệ phần tử từ nguong trở lên, làm tròn 3 chữ số.
- `thong_ke_mon(ds_hs, mon)`: trả về năm giá trị (*thấp nhất*, *cao nhất*, *trung bình*, *độ lệch chuẩn*, *tỉ lệ đạt*) của một môn, bằng cách gọi bốn hàm ở trên.

Phần `--- A2` phải in ra đúng như sau

```
--- A2
(4.0, 9.0, 7.0) 1.97
0.5 0.75
toan (0, 100, 66.09, 15.16, 0.865)
doc (17, 100, 69.17, 14.59, 0.91)
viet (10, 100, 68.05, 15.19, 0.886)
```

- Độ lệch chuẩn ra `2.27`: đã chia cho $n-1$ thay vì n.
- Dòng hai ra `0.25 0.75`: đã dùng `>` thay cho `>=`.
- Ba dòng cuối in `None`: `thong_ke_mon` tính xong nhưng thiếu `return`.

3. Tham số (30 phút)

Mục đích

Dùng tham số chỉ nhận theo tên để lọc dữ liệu, `*args` để nhận số đối số tuỳ ý, và giá trị mặc định `None` cho tham số khả biến.

3.1 Mã mẫu: `loc_hoc_sinh`

Hai tham số sau dấu `*` bắt buộc gọi theo tên. Giá trị mặc định `None` nghĩa là *không lọc theo cột đó*.

xu_ly_hs.py — mã mẫu A3

```
def loc_hoc_sinh(ds_hs, *, gioi_tinh=None, on_thi=None):
    """Loc hoc sinh theo gioi tinh va/hoac on thi; None nghia la khong loc."""
    ket_qua = []
    for hs in ds_hs:
        if gioi_tinh is not None and hs["gioi_tinh"] != gioi_tinh:
            continue
        if on_thi is not None and hs["on_thi"] != on_thi:
            continue
        ket_qua.append(hs)
    return ket_qua
```

Gọi theo vị trí `loc_hoc_sinh(ds, "female")` sẽ dừng với `TypeError: loc_hoc_sinh() takes 1 positional argument but 2 were given`.
Slide Bài 5: *Tham số chỉ nhận theo tên: dấu `*`*; *`*args`*; *Cạm bẫy lớn nhất của bài: giá trị mặc định khả biến*.

3.2 Sinh viên tự thực hiện

- `diem_tb_hs(hs, *cac_mon)`: trung bình các môn được chỉ ra của một học sinh, làm tròn 2 chữ số; gọi không kèm môn nào thì lấy cả ba môn trong `MON`.
- `danh_sach_yeu(ds_hs, nguong=40, ket_qua=None)`: thêm số thứ tự (đếm từ 1) của các học sinh có ít nhất một môn dưới `nguong` vào cuối danh sách `ket_qua`, rồi trả về `ket_qua`. Không truyền `ket_qua` thì tạo danh sách mới.

Phần `--- A3` phải in ra đúng như sau

```
--- A3
518 174 1000
72.67 72.0 73.0
51 51 [18, 60, 328, 597, 788, 981]
```

- Dòng ba ra `51 102 ...`: đã viết `ket_qua=[]`. Danh sách mặc định được tạo một lần lúc định nghĩa hàm nên lần gọi thứ hai nối tiếp vào kết quả của lần thứ nhất.
- Dừng với `ZeroDivisionError`: chưa xử lý trường hợp gọi không kèm môn nào.

4. Phạm vi biến và hàm dựng sẵn (20 phút)

Mục đích

Đọc hằng số cấu hình từ bên trong hàm, và dùng hàm dựng sẵn thay cho vòng lặp tự viết.

4.1 Mã mẫu: `dat_ca_ba`

`NGUONG_DAT` được gán ở đầu tệp; bên trong hàm chỉ *đọc* nó nên không cần truyền vào. `all` nhận một danh sách và trả về `True` khi mọi phần tử đều đúng.

xu_ly_hs.py — mã mẫu A4

```
def dat_ca_ba(hs):
    """True neu ca ba mon deu tu NGUONG_DAT tro len."""
    return all([hs["toan"] >= NGUONG_DAT, hs["doc"] >= NGUONG_DAT,
                hs["viet"] >= NGUONG_DAT])
```

Slide Bài 5: *Bên trong hàm đọc được biến bên ngoài*; *Tầng ngoài cùng: hàm dựng sẵn*; *Cạm bẫy: đặt tên biến trùng tên hàm dựng sẵn*; *Vì sao phải học kỹ chỗ này* (chữ ký của `sorted`).

4.2 Sinh viên tự thực hiện

- `co_diem_100(hs)`: `True` nếu có ít nhất một môn đạt 100. Dùng `any`.
- `top_diem(ds_hs, mon, k=5)`: `k` điểm cao nhất của môn, sắp giảm dần. Dùng `sorted` và `lay_cot`.

Phần `--- A4` phải in ra đúng như sau

```
--- A4
812 23
[100, 100, 100, 100, 100, 100, 100, 99, 99] [100, 100, 100]
```

- Số thứ hai ra 3: đã dùng `all` thay cho `any`.
- Dừng với `TypeError: sorted expected 1 argument, got 2`: đã viết `sorted(cot, True)`. `reverse` đứng sau dấu `*` trong chữ ký của `sorted`, nên phải gọi theo tên.
- Dòng hai ra `[0, 8, 18, ...]`: thiếu `reverse=True`.

5. Tách chương trình thành mô-đun (35 phút)

Mục đích

Đặt các hàm xử lý trong mô-đun `xu_ly_hs.py`, viết chương trình chính `bao_cao.py` gọi lại mô-đun đó, rồi bổ sung một bảng báo cáo mới bằng cách chia nó thành các hàm nhỏ.

5.1 Mã mẫu: tệp `bao_cao.py`

Tạo tệp mới `bao_cao.py` cùng thư mục với `xu_ly_hs.py`, gõ nội dung sau rồi chạy `python bao_cao.py`.

bao_cao.py — mã mẫu A5

```
"""Bao cao diem thi - chuong trinh chinh, goi lai mo-dun xu_ly_hs."""

import xu_ly_hs as xl


def in_dong(nhan, ds_hs, mon):
    """In mot dong bao cao cho mot nhom hoc sinh."""
    _, _, tb, dlc, tl = xl.thong_ke_mon(ds_hs, mon)
    print(f"{nhan:<22}{len(ds_hs):>6}{tb:>8.2f}{dlc:>8.2f}{tl:>8.1%}")


if __name__ == "__main__":
    ds = xl.doc_du_lieu(xl.TEP)
    print(f"{'Nhom':<22}{'So HS':>6}{'TB':>8}{'DLC':>8}{'Dat':>8}")
    in_dong("Tat ca", ds, "toan")
    in_dong("Nu", xl.loc_hoc_sinh(ds, gioi_tinh="female"), "toan")
    in_dong("Nam", xl.loc_hoc_sinh(ds, gioi_tinh="male"), "toan")
```

- `import xu_ly_hs as xl`: mọi hàm của mô-đun gọi qua tiền tố `xl.`.
- Khối thử ở cuối `xu_ly_hs.py` nằm trong `if __name__ == "__main__":` nên *không* chạy khi `bao_cao.py` import mô-đun đó.

Slide Bài 5: *Mỗi tệp `.py` là một mô-đun*; *Ba cách đó khác nhau chỗ nào*; *`if __name__ == "__main__"`*.

5.2 Sinh viên tự thực hiện

Yêu cầu mới: báo cáo thêm điểm toán *theo trình độ học vấn của phụ huynh*, mỗi nhóm một dòng, sắp theo tên nhóm. Yêu cầu được chia thành hai hàm trong `xu_ly_hs.py`:

Hàm	Nhận	Trả
`gom_nhom`	`ds_hs`, `cot`	`dict`: giá trị của cột → danh sách học sinh. Dùng `setdefault`.
`bang_theo_nhom`	`ds_hs`, `cot`, `mon`	danh sách `tuple` (*nhóm*, *số học sinh*, *trung bình*, *độ lệch chuẩn*, *tỉ lệ đạt*), sắp tăng dần theo tên nhóm. Gọi `gom_nhom` và `thong_ke_mon`.

Sau đó thêm vào cuối khối `if __name__ == "__main__":` của `bao_cao.py` một dòng trống và vòng lặp in từng `tuple` của `xl.bang_theo_nhom(ds, "hoc_van", "toan")` theo đúng định dạng cột của `in_dong`.

Phần `--- A5` của `python xu_ly_hs.py`

```
--- A5
('free/reduced', 355, 58.92, 15.14, 0.73)
('standard', 645, 70.03, 13.64, 0.94)
```

Thứ tự ra `standard` trước: chưa sắp theo tên nhóm.

`python bao_cao.py` phải in ra đúng như sau

```
Nhom                    So HS      TB     DLC     Dat
Tat ca                   1000   66.09   15.16   86.5%
Nu                        518   63.63   15.48   83.4%
Nam                       482   68.73   14.34   89.8%

associate's degree        222   67.88   15.08   87.8%
bachelor's degree         118   69.39   14.88   90.7%
high school               196   62.14   14.50   83.2%
master's degree            59   69.75   15.02   89.8%
some college              226   67.13   14.28   90.3%
some high school          179   63.50   15.88   79.9%
```

- Dừng với `ModuleNotFoundError: No module named 'xu_ly_hs'`: `bao_cao.py` không nằm cùng thư mục với `xu_ly_hs.py`, hoặc terminal không đứng ở thư mục đó.
- Trước bảng có in thêm các dòng `--- A2` …: khối thử trong `xu_ly_hs.py` đã bị đưa ra ngoài `if __name__ == "__main__":`.

Hết phần A — kiểm lại trước khi rời phòng máy

Ba điều phải đúng

1. `python xu_ly_hs.py` in ra đúng các phần từ `--- A2` đến `--- A5`; `python bao_cao.py` in ra đúng bảng ở mốc kiểm cuối.
2. Mọi hàm có một dòng docstring; không hàm nào có giá trị mặc định là `[]` hay `{}`.
3. `thong_ke_mon` và `bang_theo_nhom` gọi lại các hàm nhỏ, không chép lại thân của chúng.

Giữ lại bài làm

Chép `xu_ly_hs.py` và `bao_cao.py` ra USB hoặc lên Google Drive trước khi ra về. Phần B dùng lại các hàm trong hai tệp này.

PHẦN B — LÀM Ở NHÀ

Phần này **có tính điểm**.

6. Yêu cầu

Yêu cầu lớn

Với bộ dữ liệu `StudentsPerformance.csv`, xây dựng các hàm để lọc học sinh theo điều kiện bất kỳ, tóm tắt một danh sách điểm, lập bảng so sánh một môn giữa các nhóm, xếp hạng học sinh, và lập bảng trung bình theo hai cột cùng lúc.

Viết một tệp `<MSSV>_lab04.py` chứa **bảy hàm** dưới đây. Lấy tệp `MAU_BAI_NOP.py` trên Classroom làm khung. Năm hàm đầu dùng lại kỹ năng của phần A; hai hàm cuối là phần mở rộng.

Mỗi học sinh là một `dict` tám khoá như phần A. Hàm `doc_du_lieu` của phần A được chép vào tệp bài làm để tự thử.

Bài làm được chạy với dữ liệu khác

Các hàm sẽ được chạy với dữ liệu khác, gồm cả danh sách rỗng, nhóm có số học sinh chẵn, và nhiều học sinh bằng điểm nhau.

6.1 Năm hàm dùng lại phần A

Giữ **đúng tên hàm, đúng thứ tự tham số, và đúng các dấu `*`** như trong bảng. Sai tên thì hàm không được tìm thấy, và bài coi như bỏ trống dù thuật toán viết đúng.

Hàm	Trả về
`trung_vi(ds)`	trung vị của danh sách số: số ở giữa sau khi sắp xếp; số phần tử chẵn thì lấy trung bình hai số giữa. Danh sách rỗng → `None`. **Không được làm thay đổi `ds`.**
`tom_tat(ds)`	năm giá trị (*số phần tử*, *nhỏ nhất*, *trung vị*, *trung bình* làm tròn 2, *lớn nhất*). Danh sách rỗng → `(0, None, None, None, None)`.
`loc(ds_hs, *, mon="toan", tu=0, den=100, **dieu_kien)`	danh sách học sinh có điểm môn `mon` trong đoạn `[tu, den]` *và* khớp mọi điều kiện `cột=giá trị` trong `dieu_kien`. Ví dụ `loc(ds, den=39, gioi_tinh="female")`.
`gom_canh_bao(ds_hs, *, nguong=40, ket_qua=None)`	thêm cặp (*số thứ tự*, *tên môn*) cho mỗi môn dưới `nguong` vào cuối `ket_qua`, theo thứ tự học sinh rồi theo thứ tự `MON`; trả về `ket_qua`. Không truyền `ket_qua` thì tạo danh sách mới.
`bang_so_sanh(ds_hs, cot, *, mon="toan")`	danh sách các dòng chữ: một dòng tiêu đề, rồi mỗi nhóm của `cot` một dòng, sắp theo tên nhóm. Định dạng ở khối mã dưới bảng.

Định dạng dòng của bang_so_sanh

```
f"{'Nhom':<22}{'So HS':>6}{'TB':>8}{'Trung vi':>10}"      # dong tieu de
f"{ten:<22}{n:>6}{tb:>8.2f}{tv:>10.1f}"                   # moi nhom
```

Ví dụ: in từng dòng của bang_so_sanh(ds, "on_thi")

```
Nhom                   So HS       TB  Trung vi
completed                358    69.70      69.0
none                     642    64.08      64.0
```

Hàm	Ở lớp đã làm tại
`trung_vi`, `tom_tat`	A2 `thong_ke`, `thong_ke_mon` (trả về nhiều giá trị); A4 `sorted`
`loc`	A3 `loc_hoc_sinh`; slide Bài 5 `**kwargs`
`gom_canh_bao`	A3 `danh_sach_yeu`
`bang_so_sanh`	A5 `gom_nhom`, `bang_theo_nhom`

6.2 Hai hàm mở rộng

Phần này *chưa hướng dẫn ở lớp*.

`xep_hang(ds_hs, mon, *, k=None)`

Danh sách cặp (*số thứ tự*, *điểm*) — số thứ tự đếm từ 1 — sắp giảm dần theo điểm; nhiều học sinh bằng điểm thì số thứ tự nhỏ đứng trước. `k=None` trả về tất cả; ngược lại trả về `k` cặp đầu.
Ví dụ: điểm toán của bốn học sinh là `70, 90, 70, 95` thì kết quả là `[(4, 95), (2, 90), (1, 70), (3, 70)]`.

`bang_hai_chieu(ds_hs, cot_dong, cot_cot, *, mon="toan")`

`dict`: cặp (*giá trị của* `cot_dong`, *giá trị của* `cot_cot`) → điểm trung bình môn `mon` của các học sinh thuộc cặp đó, làm tròn 2. Chỉ có mặt những cặp có ít nhất một học sinh.

Quy định về cách viết hàm

1. Cả bảy hàm phải có docstring.
2. Không hàm nào dài quá 20 dòng, tính cả dòng `def` và docstring.
3. Không dùng `global`. Không dùng `[]`, `{}` làm giá trị mặc định.
4. `bang_so_sanh` phải gọi ít nhất hai hàm khác được định nghĩa trong cùng tệp.

Được viết thêm hàm phụ ngoài bảy hàm đã nêu tên.

7. Gợi ý

Cho năm hàm đầu

1. `trung_vi`: với danh sách đã sắp có n phần tử, vị trí giữa là `n // 2`.
2. `loc`: `dieu_kien` là một `dict`; duyệt bằng `.items()`.
3. `bang_so_sanh`: cùng cách chia bước với `bang_theo_nhom` ở A5, thay `thong_ke_mon` bằng `tom_tat`.

Cho phần mở rộng

`xep_hang`: Python sắp danh sách `tuple` theo phần tử thứ nhất, bằng nhau thì theo phần tử thứ hai. `bang_hai_chieu`: `tuple` dùng được làm khoá của `dict` (Bài thực hành 3, A3).

Lỗi không có thông báo

Trong `trung_vi`, viết `ds.sort()` thay cho `sorted(ds)` cho ra đúng trung vị, nhưng danh sách của nơi gọi hàm bị sắp lại theo: `[9, 1, 5, 3]` thành `[1, 3, 5, 9]` sau khi gọi. Phạm vi biến chặn phép gán, không chặn phép sửa (slide Bài 5, *Một ngoại lệ phải nhớ*).

8. Nộp bài

Ghi nhớ

Nộp **một tệp duy nhất** tên `<MSSV>_lab04.py` lên Classroom. Ví dụ sinh viên 24280001 nộp tệp `24280001_lab04.py`. Số bài ghi hai chữ số.
Trong tệp chứa bảy hàm theo đúng tên đã cho, các hàm phụ tự viết, và phần tự thử nghiệm — nhưng phần thử nghiệm, kể cả lệnh đọc tệp dữ liệu, phải nằm trong khối `if __name__ == "__main__"`: ở cuối tệp, nếu không nó sẽ chạy mỗi khi tệp được mở ra chấm.
Không dùng `import` nào.

Hạn nộp

24 giờ trước giờ bắt đầu ca thực hành tiếp theo của lớp.
Ví dụ ca thực hành 5 của lớp bắt đầu lúc 7g30 thứ Năm thì hạn nộp là 7g30 thứ Tư.